Vidyavardhini's

College of Engineering & Technology

Vasai Road (W)

Department of
Artificial Intelligence and Data Science

Laboratory Manual

Semester	VIII	Class	B.E.
Course Code	CSDOL8011		
Course Name	Reinforcement Learning Lab		

Vidyavardhini's College of Engineering & Technology

Vision

To be a premier institution of technical education; always aiming at becoming a valuable resource for industry and society.

Mission

- To provide a technologically inspiring environment for learning.
- To promote creativity, innovation and professional activities.
- To inculcate ethical and moral values.
- To cater personal, professional and societal needs through quality education.

Program Outcomes (POs):

Engineering Graduates will be able to:

- **PO1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- **PO2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- **PO3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- **PO4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- **PO5. Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- **PO6. The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- **PO7. Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- **PO8. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- **PO9. Individual and teamwork:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- **PO10. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- **PO11. Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- **PO12. Life-long learning:** Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Course Objectives

1	Introduce the fundamentals of reinforcement learning and problem formulation using MDPs and Bandit problems
2	Explode different exploration strategies and their impact on online leaning scenarios
3	Understand dynamic programming algorithms for solving Markov Decision Processes.
4	Apply dynamic programming techniques to solve small-scale MDP problems.
5	Implement and compare Monte Carlo methods and Temporal-Difference learning algorithms.
6	Explore real-world applications of reinforcement learning in domains such as autonomous driving or robotics.

Course Outcomes

CO	Students will be able to	Action verbs	Bloom's Level
CSDOL8011.1	Apply solid understanding of reinforcement learning concepts and problem formulation.	Apply	Apply (Level 3)
CSDOL8011.2	Apply and compare exploration strategies in online learning scenarios.	Apply	Apply (Level 3)
CSDOL8011.3	Apply Markov Decision Processes using dynamic programming algorithms	Apply	Apply (Level 3)
CSDOL8011.4	Apply and analyze Monte Carlo methods and Temporal- Difference learning algorithms	Apply	Apply (Level 3)
CSDOL8011.5	Apply Neural network or deep Q-network (DQN) to train an agent	Apply/ Analyze	Apply (Level 3)
CSDOL8011.6	Explore practical applications of reinforcement learning in real-world domains.	Understand	Understand (Level 2)

Mapping of Experiments with Course Outcomes

List of Experiments	Course Outcomes					
	CSDOL8011.1	CSDOL8011.2	CSDOL8011.3	CSDOL8011.4	CSDOL8011.5	CSDOL8011.6
Implement a simple grid-world environment and training an agent using basic Q-learning	3	-	-	-	-	-
Implement a multi-armed bandit problem and compare different exploration strategies like epsilon-greedy and UCB.	-	3	-	-	-	-
Implement a basic grid-world environment as an MDP and applying policy iteration and value iteration algorithms to find optimal policies.	-	3	-	-	-	-
Apply dynamic programming algorithms, such as policy evaluation and policy improvement, to solve a small-scale MDP problem.	-	-	3	-	-	-
Implementing Monte Carlo control and Temporal Difference (TD) learning algorithms to train an agent in a grid-world environment.	-	-		3	-	-
To implement and compare different exploration	-	-	-	3	-	-

strategies, and analyze their impact on the agent's learning performance.						
To implement value function approximation using linear regression and using neural network.	-	-	-		3	-
To implement a Deep Q-Network (DQN) for Atari Game Playing.	-	-	-	-	3	-
To implement policy gradient methods, such as REINFORCE or Proximal Policy Optimization (PPO), to train an agent in a continuous control environment.	-	-	-	-	-	3
Case Study on Applying reinforcement learning techniques to solve a real- world problem, such as training a self-driving car to navigate a simulated road environment.	-	-	-	-	-	3

Sr. No.	Name of Experiment	D.O.P.	D.O.C.	Page No.	Remark
1	Implement a simple grid-world environment and training an agent using basic Q-learning.				

2	Implement a multi-armed bandit problem and compare different exploration strategies like epsilon-greedy and UCB.				
3	Implement a basic grid-world environment as an MDP and applying policy iteration and value iteration algorithms to find optimal policies.				
4	Apply dynamic programming algorithms, such as policy evaluation and policy improvement, to solve a small-scale MDP problem.				
5	Implementing Monte Carlo control and Temporal Difference (TD) learning algorithms to train an agent in a grid-world environment.				
6	To implement and compare different exploration strategies, and analyze their impact on the agent's learning performance.				
7	To implement value function approximation using linear regression and using neural network.				
8	To implement a Deep Q-Network (DQN) for Atari Game Playing.				
9	To implement policy gradient methods, such as REINFORCE or Proximal Policy Optimization (PPO), to train an agent in a continuous control environment.				
10	Case Study on Applying reinforcement learning techniques to solve a real- world problem, such as training a self-driving car to navigate a simulated road environment.				

D.O.P: Date of performance

D.O.C : Date of correction

Experiment No. 1
Implement a simple grid-world environment and training an agent using basic Q-learning.
Date of Performance:
Date of Submission:
Marks:
Sign:

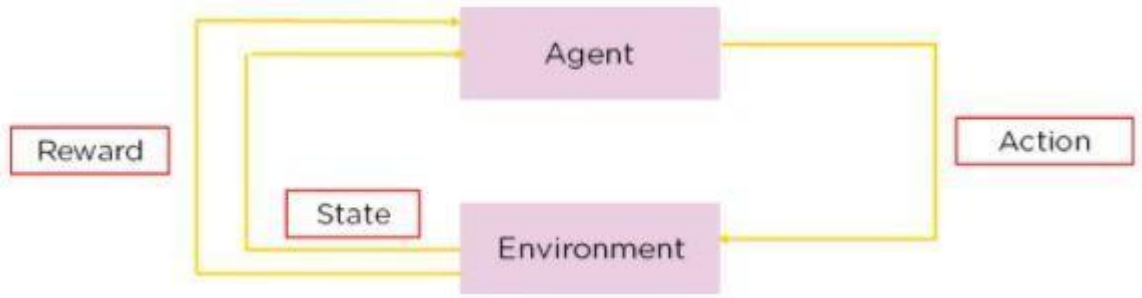
Aim: Implement a simple grid-world environment and training an agent using basic Q-learning.

Objective: To implement the Q-learning update rule and maintain a **Q-table** that stores the expected rewards for each state-action pair.

Theory:

Q-Learning

Q-Learning is a Reinforcement learning policy that will find the next best action, given a current state. It chooses this action at random and aims to maximize the reward.



Q-learning is a model-free, off-policy reinforcement learning that will find the best course of action, given the current state of the agent. Depending on where the agent is in the environment, it will decide the next action to be taken.

The objective of the model is to find the best course of action given its current state. To do this, it may come up with rules of its own or it may operate outside the policy given to it to follow. This means that there is no actual need for a policy, hence we call it off-policy.

Model-free means that the agent uses predictions of the environment's expected response to move forward. It does not use the reward system to learn, but rather, trial and error.

An example of Q-learning is an Advertisement recommendation system. In a normal ad recommendation system, the ads you get are based on your previous purchases or websites you may have visited. If you've bought a TV, you will get recommended TVs of different brands.

Using Q-learning, we can optimize the ad recommendation system to recommend products that are frequently bought together. The reward will be if the user clicks on the suggested product.

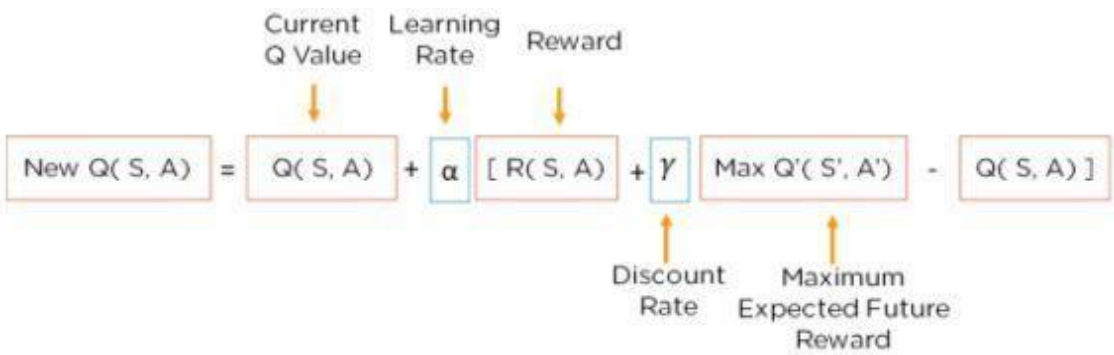
Important Terms in Q-Learning

1. States: The State, S, represents the current position of an agent in an environment.
2. Action: The Action, A, is the step taken by the agent when it is in a particular state.
3. Rewards: For every action, the agent will get a positive or negative reward.
4. Episodes: When an agent ends up in a terminating state and can't take a new action.
5. Q-Values: Used to determine how good an Action, A, taken at a particular state, S, is. Q (A, S).
6. Temporal Difference: A formula used to find the Q-Value by using the value of current state andaction and previous state and action.

What Is The Bellman Equation?

The Bellman Equation is used to determine the value of a particular state and deduce how good it is to be in/take that state. The optimal state will give us the highest optimal value.

The equation is given below. It uses the current state, and the reward associated with that state, along with the maximum expected reward and a discount rate, which determines its importance to the current state, to find the next state of our agent. The learning rate determines how fast or slow, the model will belearning.



How to Make a Q-Table?

While running our algorithm, we will come across various solutions and the agent will take multiple paths. How do we find out the best among them? This is done by tabulating our findings in a table calleda Q-Table.

A Q-Table helps us to find the best action for each state in the environment. We use the Bellman Equation at each state to get the expected future state and reward and save it in a table to compare with other states.

Lets us create a q-table for an agent that has to learn to run, fetch and sit on command. The steps taken to construct a q-table are :

Step 1: Create an initial Q-Table with all values initialized to 0

When we initially start, the values of all states and rewards will be 0. Consider the Q-Table shown below which shows a dog simulator learning to perform actions :

Action	Fetching	Sitting	Running
Start	0	0	0
Idle	0	0	0
Wrong Action	0	0	0
Correct Action	0	0	0
End	0	0	0

Step 2: Choose an action and perform it. Update values in the table

This is the starting point. We have performed no other action as of yet. Let us say that we want the agent to sit initially, which it does. The table will change to:

Action	Fetching	Sitting	Running
Start	0	1	0
Idle	0	0	0
Wrong Action	0	0	0
Correct Action	0	0	0
End	0	0	0

Step 3: Get the value of the reward and calculate the value Q-Value using Bellman Equation

For the action performed, we need to calculate the value of the actual reward and the Q(S, A) value

Action	Fetching	Sitting	Running
Start	0	1	0
Idle	0	0	0
Wrong Action	0	0	0
Correct Action	0	34	0
End	0	0	0

Step 4: Continue the same until the table is filled or an episode ends

The agent continues taking actions and for each action, the reward and Q-value are calculated and itupdates the table.

Implementation:

```
import numpy as np

import random



# Step 1: Define Grid World

grid_size = 4

num_states = grid_size * grid_size

num_actions = 4  # up, down, left, right
```

```
# Actions

actions = ['up', 'down', 'left', 'right']


# Step 2: Initialize Q-table

Q = np.zeros((num_states, num_actions))


# Step 3: Parameters

alpha = 0.1      # learning rate

gamma = 0.9      # discount factor

epsilon = 0.2    # exploration rate

episodes = 500


# Step 4: Reward function

def get_reward(state):

    if state == num_states - 1:  # goal state

        return 10

    return -1  # penalty for each step


# Step 5: State transition

def next_state(state, action):

    row, col = divmod(state, grid_size)
```

```
    if action == 0 and row > 0: row -= 1        # up
    elif action == 1 and row < grid_size - 1: row += 1  # down
    elif action == 2 and col > 0: col -= 1      # left
    elif action == 3 and col < grid_size - 1: col += 1  # right

    return row * grid_size + col

# Step 6: Training
for ep in range(episodes):
    state = 0  # start from top-left

    while state != num_states - 1:
        # Choose action (epsilon-greedy)
        if random.uniform(0,1) < epsilon:
            action = random.randint(0, num_actions-1)
        else:
            action = np.argmax(Q[state])

        new_state = next_state(state, action)
        reward = get_reward(new_state)

        # Q-learning update rule
```

```
        Q[state, action] = Q[state, action] + alpha * (
            reward + gamma * np.max(Q[new_state]) - Q[state, action]
        )

        state = new_state

# Step 7: Display Q-table
print("Q-Table:\n", Q)

# Step 8: Test learned policy
state = 0
path = [state]

while state != num_states - 1:
    action = np.argmax(Q[state])
    state = next_state(state, action)
    path.append(state)

print("Optimal Path:", path)
```

Conclusion:

Q-learning is an effective reinforcement learning algorithm for training agents in unknown environments. It helps the agent learn optimal actions through trial and error using rewards. The grid-world example demonstrates how an agent improves its decision-making over time by updating Q-values.

Experiment No. 2
Implement a multi-armed bandit problem and compare different exploration strategies like epsilon-greedy and UCB.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: To implement a Multi-Armed Bandit (MAB) problem and compare the performance of different exploration strategies such as Epsilon-Greedy and Upper Confidence Bound (UCB) in maximizing cumulative reward.

Objective:

- To understand the concept of the **Multi-Armed Bandit problem** in reinforcement learning.
- To implement the **Epsilon-Greedy strategy** for balancing exploration and exploitation.
- To implement the **Upper Confidence Bound (UCB) algorithm** for action selection.
- To compare the performance of both strategies based on:

  - Cumulative reward
  - Regret (difference from optimal reward)
  - Convergence behavior

Theory:

The Multi-Armed Bandit Problem (MAB) is a fundamental problem in the field of reinforcement learning and decision-making under uncertainty. The problem involves a gambler who has to choose among several slot machines (also called "one-armed bandits") with unknown payout probabilities. The gambler's objective is to maximize his or her total payout by choosing the best slot machine to play.
The MAB problem is commonly used in various fields, such as clinical trials, online advertising, and recommendation systems. The MAB problem can be seen as a trade-off between exploration and exploitation. Exploration refers to the gambler trying out different slot machines to learn about their payout probabilities, while exploitation refers to the gambler sticking to the slot machine that has shown the highest payout probability so far.

A bandit is defined as someone who steals your money. A one-armed bandit is a simple slot machine wherein you insert a coin into the machine, pull a lever, and get an immediate reward. But why is it called a bandit? It turns out all casinos configure these slot machines in such a way that all gamblers end up losing money!

A multi-armed bandit is a complicated slot machine wherein instead of 1, there are several levers that a gambler can pull, with each lever giving a different return. The probability distribution for the reward corresponding to each lever is different and is unknown to the gambler

Multi-Armed Bandit Problem

The Multi-Armed Bandit problem is a classic problem in reinforcement learning where an agent must choose between multiple actions (arms), each providing a stochastic reward.

- Each arm has an **unknown probability distribution** of rewards.
- The goal is to **maximize total reward over time**.
- The key challenge is the **exploration vs exploitation trade-off**:
  - **Exploration**: Trying new arms to gain information
  - **Exploitation**: Choosing the best-known arm

Epsilon-Greedy Strategy

This is a simple and widely used method:

- With probability **ε (epsilon)** → choose a random action (**exploration**)
- With probability **1 − ε** → choose the best-known action (**exploitation**)

Advantages:

- Easy to implement
- Ensures continuous exploration

Disadvantages:

- May explore too much or too little depending on ε
- Fixed ε may not adapt over time

Upper Confidence Bound (UCB)

UCB is a more advanced strategy that balances exploration and exploitation using statistical confidence.

Idea:

- Choose the action with the highest **upper confidence bound** value
- It considers:
  - Average reward of the arm
  - Uncertainty (less tried arms get higher priority)

The selection formula is:

$$A_t = \arg\max_a \left(Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right)$$

Where:

- $Q_t(a)$: Estimated value of action a
- $N_t(a)$: Number of times action $a$is selected
- t: Total number of trials
- c: Exploration parameter

Advantages:

- Smart exploration based on uncertainty
- Better theoretical performance

Disadvantages:

- Slightly more complex to implement
- Requires careful tuning of parameter c

4. Performance Comparison

To evaluate both strategies:

- **Cumulative Reward**: Total reward collected over time
- **Regret**: Loss due to not always choosing the optimal arm
- **Convergence**: How quickly the algorithm identifies the best arm

Expected Outcome:

- Epsilon-Greedy is simpler but less efficient
- UCB generally achieves **better long-term performance**

Implementation:

import numpy as np

```
import matplotlib.pyplot as plt

# Step 1: Define Bandit Environment
n_arms = 5
true_probs = [0.1, 0.5, 0.3, 0.8, 0.6]  # true reward probabilities
n_steps = 1000

def pull_arm(arm):
    return 1 if np.random.rand() < true_probs[arm] else 0

# ------------------------------
# Epsilon-Greedy Implementation
# ------------------------------
epsilon = 0.1
Q_eps = np.zeros(n_arms)
N_eps = np.zeros(n_arms)
rewards_eps = []

for t in range(n_steps):
    if np.random.rand() < epsilon:
        action = np.random.randint(n_arms)
    else:
        action = np.argmax(Q_eps)

    reward = pull_arm(action)
    N_eps[action] += 1
    Q_eps[action] += (reward - Q_eps[action]) / N_eps[action]
    rewards_eps.append(reward)

# ------------------------------
# UCB Implementation
# ------------------------------
```

```
Q_ucb = np.zeros(n_arms)
N_ucb = np.zeros(n_arms)
rewards_ucb = []

c = 2

for t in range(n_steps):
    if t < n_arms:
        action = t
    else:
        ucb_values = Q_ucb + c * np.sqrt(np.log(t+1) / (N_ucb + 1e-5))
        action = np.argmax(ucb_values)

    reward = pull_arm(action)
    N_ucb[action] += 1
    Q_ucb[action] += (reward - Q_ucb[action]) / N_ucb[action]
    rewards_ucb.append(reward)

# Step 3: Compute cumulative rewards
cum_rewards_eps = np.cumsum(rewards_eps)
cum_rewards_ucb = np.cumsum(rewards_ucb)

# Step 4: Plot comparison
plt.plot(cum_rewards_eps, label="Epsilon-Greedy")
plt.plot(cum_rewards_ucb, label="UCB")
plt.xlabel("Steps")
plt.ylabel("Cumulative Reward")
plt.title("Epsilon-Greedy vs UCB")
plt.legend()
plt.show()
```

Conclusion:

The Multi-Armed Bandit problem demonstrates the importance of balancing exploration and exploitation.

- **Epsilon-Greedy** is simple and easy to implement but may explore inefficiently.
- **UCB** provides smarter exploration by considering uncertainty and performs better in the long run.
  Thus, UCB is generally more effective in maximizing cumulative reward.

Experiment No. 3
Implement a basic grid-world environment as an MDP and applying policy iteration and value iteration algorithms to find optimal policies.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Implementing a basic grid-world environment as an MDP and applying policy iteration and value iteration algorithms to find optimal policies.

Objective: The objective is to create a simplified grid-world environment, representing it as a Markov Decision Process (MDP), and then employ policy iteration and value iteration algorithms to compute optimal policies within this environment, facilitating a fundamental understanding of dynamic programming techniques in reinforcement learning.

Theory:

Markov Decision Process (MDP):

1. An MDP is a mathematical framework used to model decision-making in environments where outcomes are partially random and partially under the control of an agent.
2. It consists of a set of states, a set of actions, transition probabilities, rewards, and a discount factor.
3. In a grid-world environment, each cell represents a state, and the agent can take actions (move up, down, left, right) to transition between states.

Optimal Policies:

4. An optimal policy specifies the best action to take in each state to maximize cumulative rewards over time.
5. Finding optimal policies involves determining the best strategy for the agent to navigate the environment and achieve its goals.

Policy Iteration:

6. Policy iteration is an iterative algorithm for finding the optimal policy in an MDP.
7. It alternates between two steps: policy evaluation and policy improvement.
8. Policy evaluation involves estimating the value function for a given policy, while policy improvement updates the policy based on the current value function.
9. This process continues until the policy converges to an optimal policy.

Algorithm for Policy Iteration:

1. Policy Evaluation:

   - Initialize V(s) arbitrarily for all states

   - Repeat until $\Delta < \varepsilon$ (small positive number):

- $\Delta \leftarrow 0$
- For each state s:
  - $v \leftarrow V(s)$
  - $V(s) \leftarrow \Sigma(P(s' \mid s, \pi(s)) * [R(s, \pi(s), s') + \gamma * V(s')])$
  - $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

2. Policy Improvement:
   - Policy_stable ← true
   - For each state s:
     - old_action ← $\pi(s)$
     - $\pi(s) \leftarrow \text{argmax}[a]\{\Sigma(P(s' \mid s, a) * [R(s, a, s') + \gamma * V(s')])\}$
     - If old_action $\neq \pi(s)$, then Policy_stable ← false

If Policy_stable, then stop and return optimal policy π^*

Value Iteration:

10. Value iteration is another iterative algorithm used to find optimal policies in MDPs.
11. It iteratively updates the value function for each state until it converges to the optimal value function.
12. The optimal policy can then be derived from the optimal value function by selecting actions that maximize expected returns.

Algorithm of Value Iteration:

Initialize V(s) arbitrarily for all states

Repeat until $\Delta < \varepsilon$ (small positive number):

- $\Delta \leftarrow 0$

For each state s:

- $v \leftarrow V(s)$
- $V(s) \leftarrow \max[a]\{\Sigma(P(s' \mid s, a) * [R(s, a, s') + \gamma * V(s')])\}$
- $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

Return optimal policy π^* such that $\pi^*(s) = \text{argmax}[a]\{\Sigma(P(s' \mid s, a) * [R(s, a, s') + \gamma * V(s')])\}$

In these algorithms:

1. **π** represents the policy, which specifies the action to be taken in each state.
2. **V(s)** represents the value function, which estimates the expected cumulative reward starting from state **s** and following the current policy **π**.
3. **P(s' | s, a)** represents the transition probability from state **s** to state **s'** under action **a**.
4. **R(s, a, s')** represents the reward obtained when transitioning from state **s** to state **s'** under action **a**.
5. **γ** is the discount factor, representing the importance of future rewards compared to immediate rewards.
6. **Δ** is a small positive number used to determine convergence.

These algorithms iteratively update the value function and policy until convergence, where the policy either stabilizes (in policy iteration) or the value function converges (in value iteration). The resulting policy is then considered optimal for the given grid-world environment.

Implementation:

```
import numpy as np


# Step 1: Define Grid World

grid_size = 4

num_states = grid_size * grid_size

actions = [0, 1, 2, 3]  # up, down, left, right

gamma = 0.9


# Step 2: Transition function

def next_state(s, a):

    row, col = divmod(s, grid_size)
```

```
    if a == 0 and row > 0: row -= 1

    elif a == 1 and row < grid_size - 1: row += 1

    elif a == 2 and col > 0: col -= 1

    elif a == 3 and col < grid_size - 1: col += 1



    return row * grid_size + col



# Reward function

def reward(s):

    return 10 if s == num_states - 1 else -1



# ----------------------------

# POLICY ITERATION

# ----------------------------

V = np.zeros(num_states)

policy = np.zeros(num_states, dtype=int)



def policy_evaluation(policy, V):

    theta = 1e-4

    while True:

        delta = 0

        for s in range(num_states):
```

```
            v = V[s]
            a = policy[s]
            s_next = next_state(s, a)
            V[s] = reward(s_next) + gamma * V[s_next]
            delta = max(delta, abs(v - V[s]))
        if delta < theta:
            break
    return V


def policy_improvement(V, policy):
    stable = True
    for s in range(num_states):
        old_action = policy[s]
        q_values = []
        for a in actions:
            s_next = next_state(s, a)
            q_values.append(reward(s_next) + gamma * V[s_next])
        policy[s] = np.argmax(q_values)
        if old_action != policy[s]:
            stable = False
    return policy, stable
```

```
# Run Policy Iteration

while True:

    V = policy_evaluation(policy, V)

    policy, stable = policy_improvement(V, policy)

    if stable:

        break


print("Optimal Policy (Policy Iteration):")

print(policy.reshape(grid_size, grid_size))


# ----------------------------

# VALUE ITERATION

# ----------------------------

V_vi = np.zeros(num_states)


theta = 1e-4

while True:

    delta = 0

    for s in range(num_states):

        v = V_vi[s]

        q_values = []

        for a in actions:
```

```
            s_next = next_state(s, a)
            q_values.append(reward(s_next) + gamma * V_vi[s_next])
        V_vi[s] = max(q_values)
        delta = max(delta, abs(v - V_vi[s]))
    if delta < theta:
        break

# Derive Policy
policy_vi = np.zeros(num_states, dtype=int)
for s in range(num_states):
    q_values = []
    for a in actions:
        s_next = next_state(s, a)
        q_values.append(reward(s_next) + gamma * V_vi[s_next])
    policy_vi[s] = np.argmax(q_values)

print("\nOptimal Policy (Value Iteration):")
print(policy_vi.reshape(grid_size, grid_size))
```

Conclusion:

Difference between Value Iteration and Policy Iteration:

Feature	Policy Iteration	Value Iteration
Approach	Alternates between evaluation & improvement	Directly updates value function
Speed	Fewer iterations but costly per iteration	More iterations but faster per step
Policy	Explicitly updated	Derived after value convergence
Complexity	Higher per iteration	Simpler

Main Steps of Value Iteration:

1. Initialize value function
2. Update value using Bellman optimality equation
3. Repeat until convergence
4. Derive optimal policy from value function

Experiment No. 4
Apply dynamic programming algorithms, such as policy evaluation and policy improvement, to solve a small-scale MDP problem.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Applying dynamic programming algorithms, such as policy evaluation and policy improvement, to solve a small-scale MDP problem.

Objective: The objective is to employ dynamic programming algorithms, such as policy evaluation and policy improvement, to effectively address a small-scale Markov Decision Process (MDP) problem, aiming to iteratively refine and optimize policies, thereby gaining insights into foundational concepts of reinforcement learning and optimization.

Theory:

Markov Decision Processes (MDPs): MDPs are mathematical frameworks used to model decision-making problems where outcomes are partly random and partly under the control of a decision-maker. They consist of states, actions, transition probabilities, rewards, and a discount factor.

Policy Evaluation: Policy evaluation is the process of determining the value function for a given policy. The value function represents the expected cumulative reward starting from a particular state and following a specific policy. The basic idea is to iteratively update the value of each state until convergence.

Algorithm for policy evaluation :

Input:
MDP: (S, A, P, R, γ), where
S is the set of states.
A is the set of actions.
P is the transition probability matrix, P(s' | s, a), representing the probability of transitioning to state s' from state s by taking action a.
R is the reward function, R(s, a, s'), representing the immediate reward received after transitioning from state s to state s' by taking action a.
γ is the discount factor.

Output:
Value function V(s) for each state s.

Algorithm:
Initialize the value function arbitrarily: V(s) for all s in S.
Repeat until convergence:
Initialize Δ to be a very large value (e.g., infinity).

For each state s in S:

Let v be the current value of V(s).
Update V(s) using the Bellman expectation equation:
$V(s) = \Sigma [P(s' | s, a) * (R(s, a, s') + \gamma * V(s'))]$ over all possible next states s' and actions a.

Update Δ to be the maximum between Δ and $|v - V(s)|$.
If Δ is smaller than a predefined threshold ε, break.
Return the converged value function V(s).

Explanation:

The algorithm iteratively updates the value function V(s) for each state s using the Bellman expectation equation until convergence.
At each iteration, it computes the expected value of being in state s and taking action a, which is the sum of immediate reward R(s, a, s') and the discounted value of the next state V(s').
The process continues until the maximum change in the value function between iterations (Δ) falls below a predefined threshold ε, indicating convergence.
The output is the converged value function V(s), which represents the expected cumulative reward from each state under the given policy.
This algorithm is known as "Iterative Policy Evaluation" and is a fundamental component of dynamic programming approaches for solving MDPs. It provides a way to estimate the value of each state under a given policy.

Policy Improvement: Policy improvement involves selecting better actions in each state to improve the current policy. It's based on the idea of greedily selecting actions that maximize the expected cumulative reward given the current value function.

Algorithm for policy improvement:
Input:

MDP: (S, A, P, R, γ), where
S is the set of states.
A is the set of actions.
P is the transition probability matrix, P(s' | s, a), representing the probability of transitioning to state s' from state s by taking action a.
R is the reward function, R(s, a, s'), representing the immediate reward received after transitioning from state s to state s' by taking action a.
γ is the discount factor.
Value function V(s) for each state s.

Output:

Improved policy π'(s) for each state s.

Algorithm:

Initialize a boolean variable policy_stable to true.
For each state s in S, do:

Let old_action be the current action selected by the policy π for state s.
Compute Q-value for each action a in A:
Q(s, a) = Σ [P(s' | s, a) * (R(s, a, s') + γ * V(s'))] over all possible next states s'.
Select the action a_max that maximizes the Q-value: a_max = argmax(Q(s, a)).
Update the policy π'(s) to select the action a_max.
Check for policy stability:

If the new policy π' is different from the old policy π for any state s, set policy_stable to false.
If policy_stable is true, return the improved policy π'.
Else, return to step 2 and repeat the process with the updated policy π'.

Explanation:

The algorithm iterates over each state in the state space and computes the Q-value for each action based on the current value function V(s).
It selects the action that maximizes the Q-value as the new action for the state.
The process continues until the policy stabilizes, i.e., the new policy is the same as the old policy for all states.
The output is the improved policy π' that greedily selects actions to maximize the expected cumulative reward according to the current value function V(s).
This algorithm is known as "Policy Improvement" and is used in combination with policy evaluation to iteratively improve the policy until convergence to an optimal policy in dynamic programming approaches for solving MDPs.

7. **Iterative Policy Evaluation and Policy Improvement**: The process of policy evaluation and policy improvement is often interleaved. After evaluating a policy, we improve it by selecting better actions based on the updated value function. Then, we re-evaluate the policy to refine the value estimates further.

8. **Convergence**: Both policy evaluation and policy improvement converge to the optimal value function and policy if executed iteratively until convergence.

9. **Implementation**: Dynamic programming algorithms can be implemented efficiently using programming languages like Python, where you can define MDPs, transition probabilities, rewards, value functions, and policies, and then iteratively update them until convergence.

This general approach can be applied to small-scale MDP problems to find the optimal policy efficiently. However, for larger problems, approximate methods like reinforcement learning techniques may be more practical due to the computational complexity of dynamic

programming algorithms.

Implementation:

```
import numpy as np

# Step 1: Define environment
grid_size = 3
num_states = grid_size * grid_size
actions = [0, 1, 2, 3]  # up, down, left, right
gamma = 0.9

# Step 2: Transition function
def next_state(s, a):
    row, col = divmod(s, grid_size)

    if a == 0 and row > 0: row -= 1
    elif a == 1 and row < grid_size - 1: row += 1
    elif a == 2 and col > 0: col -= 1
    elif a == 3 and col < grid_size - 1: col += 1

    return row * grid_size + col

# Reward function
def reward(s):
    return 10 if s == num_states - 1 else -1

# Step 3: Initialize
V = np.zeros(num_states)
```

```
policy = np.random.choice(actions, size=num_states)


# -------------------------
# Policy Evaluation
# -------------------------
def policy_evaluation(policy, V):
    theta = 1e-4
    while True:
        delta = 0
        for s in range(num_states):
            v = V[s]
            a = policy[s]
            s_next = next_state(s, a)
            V[s] = reward(s_next) + gamma * V[s_next]
            delta = max(delta, abs(v - V[s]))
        if delta < theta:
            break
    return V


# -------------------------
# Policy Improvement
# -------------------------
def policy_improvement(V, policy):
    stable = True
    for s in range(num_states):
        old_action = policy[s]


        q_values = []
```

```
        for a in actions:
            s_next = next_state(s, a)
            q_values.append(reward(s_next) + gamma * V[s_next])

        best_action = np.argmax(q_values)
        policy[s] = best_action

        if old_action != best_action:
            stable = False

    return policy, stable

# Step 4: Policy Iteration Loop
while True:
    V = policy_evaluation(policy, V)
    policy, stable = policy_improvement(V, policy)
    if stable:
        break

# Step 5: Display Results
print("Optimal Value Function:")
print(V.reshape(grid_size, grid_size))

print("\nOptimal Policy:")
print(policy.reshape(grid_size, grid_size))
```

Conclusion:

1. Example of Dynamic Programming:

A common example is **shortest path finding** (like Dijkstra's algorithm), where the solution is built using solutions of smaller subproblems.

---

2. Dynamic Programming in Reinforcement Learning:

Dynamic Programming is used in reinforcement learning to:

- Compute value functions (policy evaluation)
- Improve policies (policy improvement)
- Find optimal strategies (policy iteration & value iteration)

It works by breaking problems into smaller subproblems and solving them iteratively using Bellman equations.

Experiment No. 5
Implementing Monte Carlo control and Temporal Difference (TD) learning algorithms to train an agent in a grid-world environment..
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Implementing Monte Carlo control and Temporal Difference (TD) learning algorithms to train an agent in a grid-world environment

Theory:

Monte Carlo Control

Monte Carlo methods are a class of algorithms used in reinforcement learning for estimating value functions and finding optimal policies. Unlike dynamic programming methods, Monte Carlo methods do not require a model of the environment and instead learn directly from experience by interacting with the environment.

Monte Carlo control combines Monte Carlo prediction with policy improvement to find the optimal policy directly from experience. After estimating the value function, the policy is improved by selecting actions that maximize the value function estimates.

Monte Carlo methods learn from complete episodes of experience. Here's the algorithm for Monte Carlo Control:

Input:

MDP: (S, A, P, R, γ), where

- S is the set of states.
- A is the set of actions.
- P is the transition probability matrix.
- R is the reward function.
- γ is the discount factor.

Number of episodes **num_episodes**.

Exploration rate **epsilon**.

Algorithm:

Optimal policy π.

Initialize:

Q-function: Q(s, a) for all s in S and a in A.

State-action visit count: N(s, a) for all s in S and a in A.

Policy π arbitrarily.

For each episode **from 1 to num_episodes**:

Generate an episode following π: (s1, a1, r1), (s2, a2, r2), ..., (sT, aT, rT).

Set G = 0.

For each time step **t** from T down to 1**:

Update return G: G = γ * G + rt+1.

If (st, at) has not been visited in this episode before:

Increment visit count: N(st, at) += 1.

Update action-value function: Q(st, at) = Q(st, at) + (1/N(st, at)) * (G - Q(st, at)).

Update policy: π(st) = argmax(Q(st, a)) for all a in A.

Return the optimal policy π.

Temporal Difference (TD) Learning

Temporal Difference learning methods update estimates based on the difference between current and future estimates. Here's the algorithm for TD Learning:

Input:

MDP: (S, A, P, R, γ), where

S is the set of states.

A is the set of actions.

P is the transition probability matrix.

R is the reward function.

γ is the discount factor.

Number of episodes **num_episodes**.

Step size **alpha**.

Exploration rate **epsilon**.

Algorithm:

Optimal policy π.

Initialize:

Q-function: Q(s, a) for all s in S and a in A.

Policy π arbitrarily.

For each episode **from 1 to num_episodes**:

Initialize state s as the starting state.

While s is not a terminal state:

Choose action a using policy derived from Q (e.g., ε-greedy).

Take action a, observe reward r and next state s'.

Update action-value function: $Q(s, a) = Q(s, a) + \alpha * (r + \gamma * \max(Q(s', a')) - Q(s, a))$.

Update policy: $\pi(s) = \text{argmax}(Q(s, a))$ for all a in A.

Set s = s'.

Return the optimal policy π.

These are high-level algorithms. The specific implementation details and additional considerations (like exploration strategy, eligibility traces, etc.) might vary based on the problem and requirements.

Implementation:

```
import numpy as np

import random


# Step 1: Grid World

grid_size = 4

num_states = grid_size * grid_size

actions = [0, 1, 2, 3]  # up, down, left, right


def next_state(s, a):

    row, col = divmod(s, grid_size)

    if a == 0 and row > 0: row -= 1

    elif a == 1 and row < grid_size - 1: row += 1

    elif a == 2 and col > 0: col -= 1

    elif a == 3 and col < grid_size - 1: col += 1

    return row * grid_size + col
```

```
def reward(s):
    return 10 if s == num_states - 1 else -1


gamma = 0.9
epsilon = 0.1
episodes = 500


# ------------------------
# MONTE CARLO CONTROL
# ------------------------
Q_mc = np.zeros((num_states, len(actions)))
returns = {}


for ep in range(episodes):
    episode = []
    state = 0


    # Generate episode
    while state != num_states - 1:
        if random.random() < epsilon:
```

```
            action = random.choice(actions)
        else:
            action = np.argmax(Q_mc[state])

        next_s = next_state(state, action)
        r = reward(next_s)
        episode.append((state, action, r))
        state = next_s

    G = 0
    visited = set()

    # Backward update
    for (s, a, r) in reversed(episode):
        G = gamma * G + r
        if (s, a) not in visited:
            visited.add((s, a))
            if (s, a) not in returns:
                returns[(s, a)] = []
            returns[(s, a)].append(G)
            Q_mc[s, a] = np.mean(returns[(s, a)])
```

```
# ------------------------
# TEMPORAL DIFFERENCE (Q-Learning)
# ------------------------
Q_td = np.zeros((num_states, len(actions)))
alpha = 0.1

for ep in range(episodes):
    state = 0

    while state != num_states - 1:
        if random.random() < epsilon:
            action = random.choice(actions)
        else:
            action = np.argmax(Q_td[state])

        next_s = next_state(state, action)
        r = reward(next_s)

        # TD Update
        Q_td[state, action] += alpha * (
```

```
        r + gamma * np.max(Q_td[next_s]) - Q_td[state, action]
    )

    state = next_s

# Step 3: Display Policies
policy_mc = np.argmax(Q_mc, axis=1)
policy_td = np.argmax(Q_td, axis=1)

print("Policy (Monte Carlo):")
print(policy_mc.reshape(grid_size, grid_size))

print("\nPolicy (TD Learning):")
print(policy_td.reshape(grid_size, grid_size))
```

Conclusion:

1. Difference between Monte Carlo and TD Learning:

Feature	Monte Carlo	TD Learning
Learning	After complete episode	Step-by-step
Update	Uses actual return	Uses estimated future reward
Speed	Slower	Faster
Requirement	Needs episode end	Works in ongoing tasks

---

2. Comparison (Advantages & Limitations):

Monte Carlo Control:

Advantages:

- Simple to understand
- Uses actual rewards (no approximation)

Limitations:

- Requires complete episodes
- Slow convergence

Temporal Difference (TD) Learning:

Advantages:

- Faster learning
- Works in real-time (online learning)
- Efficient for large problems

Limitations:

- Uses approximations (may introduce bias)
- Slightly complex

Experiment No. 6
To implement and compare different exploration strategies and analyze their impact on the agent's learning performance.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: To implement and compare different exploration strategies in a reinforcement learning setting and analyze their impact on the agent's learning performance.

Objectives:

1. To understand the importance of **exploration strategies** in decision-making problems.
2. To implement common exploration techniques such as:
   - **Epsilon-Greedy**
   - **Upper Confidence Bound (UCB)**
3. To study the **exploration vs exploitation trade-off** in learning agents.

Theory

1. Exploration vs Exploitation

In reinforcement learning, an agent must choose between:

- **Exploration**: Trying new actions to gain more knowledge
- **Exploitation**: Using known information to maximize reward

Balancing these two is crucial for optimal learning.

2. Exploration Strategies

a) Epsilon-Greedy Strategy

- The agent selects a random action with probability **ε** (exploration).
- The agent selects the best-known action with probability **1 − ε** (exploitation).

Key Idea:
Ensures exploration but may not be optimal if ε is fixed.

b) Upper Confidence Bound (UCB)

- Selects actions based on both **estimated reward** and **uncertainty**.
- Gives preference to less-explored actions using a confidence term.

$$A_t = \arg\max_a \left(Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right)$$

Key Idea:
Balances exploration intelligently by considering confidence intervals.

3. Learning Performance Metrics

To analyze the effectiveness of strategies:

- **Cumulative Reward**: Total reward obtained over time
- **Regret**: Difference between actual reward and optimal reward
- **Convergence Rate**: Speed at which the agent learns the best action

4. Impact on Learning Performance

- Proper exploration helps the agent **discover optimal actions faster**.
- Poor exploration may lead to **suboptimal decisions**.
- Advanced methods like UCB typically provide **better long-term performance** compared to simple methods like epsilon-greedy.

Implementation:

```
import numpy as np

import matplotlib.pyplot as plt



# Step 1: Environment

n_arms = 5

true_probs = [0.2, 0.5, 0.75, 0.6, 0.4]  # true reward probabilities

n_steps = 1000



def pull_arm(arm):

    return 1 if np.random.rand() < true_probs[arm] else 0
```

```
optimal_reward = max(true_probs)


# -------------------------
# Epsilon-Greedy
# -------------------------
epsilon = 0.1
Q_eps = np.zeros(n_arms)
N_eps = np.zeros(n_arms)

rewards_eps = []
regret_eps = []

for t in range(n_steps):
    if np.random.rand() < epsilon:
        action = np.random.randint(n_arms)
    else:
        action = np.argmax(Q_eps)

    reward = pull_arm(action)
    N_eps[action] += 1
    Q_eps[action] += (reward - Q_eps[action]) / N_eps[action]

    rewards_eps.append(reward)
```

```
    regret_eps.append(optimal_reward - reward)


# -------------------------
# UCB Strategy
# -------------------------
Q_ucb = np.zeros(n_arms)
N_ucb = np.zeros(n_arms)


rewards_ucb = []
regret_ucb = []


c = 2


for t in range(n_steps):
    if t < n_arms:
        action = t
    else:
        ucb_values = Q_ucb + c * np.sqrt(np.log(t+1) / (N_ucb + 1e-5))
        action = np.argmax(ucb_values)


    reward = pull_arm(action)
    N_ucb[action] += 1
    Q_ucb[action] += (reward - Q_ucb[action]) / N_ucb[action]
```

```
    rewards_ucb.append(reward)

    regret_ucb.append(optimal_reward - reward)


# -------------------------

# Cumulative Reward

# -------------------------

cum_rewards_eps = np.cumsum(rewards_eps)

cum_rewards_ucb = np.cumsum(rewards_ucb)


# -------------------------

# Plot Results

# -------------------------

plt.figure(figsize=(10,5))


plt.subplot(1,2,1)

plt.plot(cum_rewards_eps, label="Epsilon-Greedy")

plt.plot(cum_rewards_ucb, label="UCB")

plt.title("Cumulative Reward")

plt.legend()


plt.subplot(1,2,2)

plt.plot(np.cumsum(regret_eps), label="Epsilon-Greedy")

plt.plot(np.cumsum(regret_ucb), label="UCB")

plt.title("Cumulative Regret")
```

plt.legend()

plt.show()

Conclusion:

1. Which Strategy Gives Optimal Solution?

- **UCB (Upper Confidence Bound)** generally provides better and more optimal solutions.
- It balances exploration and exploitation more effectively than Epsilon-Greedy.

---

2. Advantages and Disadvantages

Epsilon-Greedy

Advantages:

- Simple to implement
- Guarantees exploration

Disadvantages:

- Fixed ε may lead to too much or too little exploration
- Slower convergence
- Random exploration is inefficient

UCB

Advantages:

- Smart exploration using uncertainty
- Faster convergence
- Better long-term performance

Disadvantages:

- Slightly complex to implement
- Requires tuning of parameter *c*

Experiment No. 7
To implement value function approximation using linear regression and using neural network.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: To implement value function approximation using **Linear Regression** and **Neural Networks**, and analyze their effectiveness in estimating value functions in reinforcement learning.

Objectives:

1. To understand the concept of **value function approximation** in reinforcement learning.
2. To implement **Linear Regression** as a function approximator for value estimation.
3. To implement a **Neural Network model** for approximating value functions..

Theory:

1. Value Function in Reinforcement Learning

In reinforcement learning, the **value function** estimates how good a state (or state-action pair) is in terms of expected future rewards.

- **State Value Function**:
  $V(s)$= Expected return starting from state s
- **Action Value Function**:
  $Q(s,a)$= Expected return for taking action $a$in state s

These functions guide the agent in selecting optimal actions.

2. Need for Function Approximation

In real-world problems:

- State space can be **very large or continuous**
- Storing values in a table (**tabular method**) becomes impractical

Solution:
Use function approximation to estimate value functions:

$$V(s) \approx \hat{V}(s,\theta)$$

Where:

- θ= model parameters
- $\hat{V}(s,\theta)$= approximated value

3. Linear Regression for Value Function Approximation

Concept

- Value function is approximated as a **linear combination of features**:

$$\hat{V}(s) = w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

or

$$\hat{V}(s) = \mathbf{w}^T\mathbf{x}$$

Where:

- **x**= feature vector representing state
- **w**= weight parameters

Working

1. Extract features from state
2. Predict value using linear model
3. Update weights using gradient descent

Advantages

- Simple and fast
- Easy to interpret

4. Neural Network for Value Function Approximation

Concept

Neural networks approximate value functions using **non-linear transformations**.

$$\hat{V}(s) = f(s, \theta)$$

Where:

- f is a neural network
- θ includes weights and biases

Working

1. Input: State representation
2. Hidden layers: Learn complex patterns
3. Output: Estimated value function

Advantages

- Can model **complex and non-linear relationships**
- Suitable for high-dimensional inputs

5. Learning Process

Both methods use:

- **Loss Function**: Mean Squared Error (MSE)

$$L = (V(s) - \hat{V}(s))^2$$

- **Optimization**:
  - Gradient Descent
  - Backpropagation (for neural networks)

6. Comparison of Linear Regression and Neural Networks

Feature	Linear Regression	Neural Network
Model Type	Linear	Non-linear
Complexity	Low	High
Training Time	Fast	Slower
Accuracy	Limited	High (for complex tasks)
Interpretability	High	Low

Implementation:

```
import numpy as np
from sklearn.linear_model import LinearRegression
import tensorflow as tf
from tensorflow.keras import layers
import matplotlib.pyplot as plt
```

```
# Step 1: Create sample states (grid world)
states = np.array([[i, j] for i in range(5) for j in range(5)])

# True value function (example: distance from goal)
goal = np.array([4, 4])
values = -np.linalg.norm(states - goal, axis=1)

# -------------------------
# Linear Regression Model
# -------------------------
lin_model = LinearRegression()
lin_model.fit(states, values)

lin_predictions = lin_model.predict(states)

# -------------------------
# Neural Network Model
# -------------------------
nn_model = tf.keras.Sequential([
    layers.Dense(16, activation='relu', input_shape=(2,)),
    layers.Dense(16, activation='relu'),
    layers.Dense(1)
])

nn_model.compile(optimizer='adam', loss='mse')

nn_model.fit(states, values, epochs=100, verbose=0)

nn_predictions = nn_model.predict(states).flatten()

# -------------------------
# Plot comparison
```

```
# -------------------------
plt.plot(values, label="True Values")
plt.plot(lin_predictions, label="Linear Regression")
plt.plot(nn_predictions, label="Neural Network")
plt.legend()
plt.title("Value Function Approximation")
plt.show()
```

Conclusion:

Value function approximation is essential in reinforcement learning when dealing with large or continuous state spaces.

- **Linear Regression** is simple, fast, and interpretable but limited to linear relationships.
- **Neural Networks** provide better accuracy and can model complex patterns but require more training and computational resources.

Thus, neural networks are more suitable for complex environments, while linear models are useful for simple problems.

Experiment No. 8
To implement a Deep Q-Network (DQN) for Atari Game Playing.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Deep Reinforcement Learning: Implementing a deep Q-network (DQN) to train an agent to play a popular Atari game

Objective: the goal is to develop an AI agent capable of autonomously learning and executing effective strategies to achieve high scores and proficient gameplay in complex Atari game environments.

Theory:

Implementing a Deep Q-Network (DQN) to train an agent to play Atari games like Pong or Space Invaders is a popular application of reinforcement learning. Here's a high-level overview of how you can do this:

2. Environment Setup:
    - Install the necessary libraries like Gym Atari, which provides an interface to Atari games.
    - Set up your Python environment with libraries like TensorFlow or PyTorch for deep learning.
3. Understanding the Environment:
    - Choose an Atari game (e.g., Pong, Space Invaders) and understand its state space, action space, and reward structure.
4. Deep Q-Network (DQN) Implementation:
    - Define a neural network architecture for your DQN. Typically, a convolutional neural network (CNN) is used to process game frames.
    - Implement the Q-network, which takes the game state as input and outputs Q-values for each action.
    - Implement an experience replay buffer to store past experiences (state, action, reward, next state).
    - Implement the epsilon-greedy strategy for action selection to balance exploration and exploitation.
    - Define the loss function, typically the mean squared error between predicted Q-values and target Q-values.
    - Train the DQN by sampling batches of experiences from the replay buffer and updating the Q-network parameters using gradient descent.
5. Training Process:

- Initialize the DQN with random weights.
- Interact with the environment using the current policy (epsilon-greedy) and collect experiences.
- Store experiences (state, action, reward, next state) in the replay buffer.
- Sample batches of experiences from the replay buffer and perform a gradient descent step to update the Q-network parameters.
- Update the target Q-network periodically to stabilize training.

6. Evaluation:
    - Periodically evaluate the performance of the trained agent by running it in the environment and measuring its average reward or other performance metrics.
    - Adjust hyperparameters and tweak the algorithm as needed based on performance.
7. Fine-tuning:
    - Experiment with different hyperparameters, neural network architectures, and training strategies to improve the agent's performance.
    - Consider advanced techniques like double DQN, dueling DQN, prioritized experience replay, or rainbow DQN for further improvements.
8. Deployment:
    - Once the agent achieves satisfactory performance, deploy it to play the game autonomously or integrate it into a larger system.

Remember that training DQN can be computationally intensive and may require significant time and computational resources, especially for complex Atari games. Be patient and monitor the training process closely to diagnose any issues and make necessary adjustments. Additionally, consider leveraging cloud computing resources or distributed training techniques to speed up the training process.

Implementation:

```
import gym
import numpy as np
import tensorflow as tf
```

```
from tensorflow.keras import layers
import random
from collections import deque

# Step 1: Load Environment
env = gym.make("CartPole-v1")  # simpler alternative to Atari for lab

state_size = env.observation_space.shape[0]
action_size = env.action_space.n

# Step 2: Hyperparameters
gamma = 0.95
epsilon = 1.0
epsilon_min = 0.01
epsilon_decay = 0.995
learning_rate = 0.001
batch_size = 32
memory = deque(maxlen=2000)

# Step 3: Build DQN Model
def build_model():
    model = tf.keras.Sequential([
        layers.Dense(24, activation='relu', input_shape=(state_size,)),
        layers.Dense(24, activation='relu'),
        layers.Dense(action_size, activation='linear')
    ])
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),
                  loss='mse')
    return model

model = build_model()
```

```
# Step 4: Store Experience
def remember(state, action, reward, next_state, done):
    memory.append((state, action, reward, next_state, done))

# Step 5: Choose Action
def act(state):
    global epsilon
    if np.random.rand() <= epsilon:
        return random.randrange(action_size)
    q_values = model.predict(state, verbose=0)
    return np.argmax(q_values[0])

# Step 6: Replay (Training)
def replay():
    global epsilon
    if len(memory) < batch_size:
        return

    minibatch = random.sample(memory, batch_size)

    for state, action, reward, next_state, done in minibatch:
        target = reward
        if not done:
            target += gamma * np.amax(model.predict(next_state, verbose=0)[0])

        target_f = model.predict(state, verbose=0)
        target_f[0][action] = target

        model.fit(state, target_f, epochs=1, verbose=0)

    if epsilon > epsilon_min:
        epsilon *= epsilon_decay
```

```
# Step 7: Training Loop
episodes = 100

for e in range(episodes):
    state = env.reset()[0]
    state = np.reshape(state, [1, state_size])

    for time in range(500):
        action = act(state)
        next_state, reward, done, _, _ = env.step(action)

        next_state = np.reshape(next_state, [1, state_size])
        remember(state, action, reward, next_state, done)

        state = next_state
        replay()

        if done:
            print(f"Episode: {e+1}/{episodes}, Score: {time}, Epsilon: {epsilon:.2f}")
            break
```

`

Conclusion:

1. What is DQN and how is it different from Q-learning?

- **DQN (Deep Q-Network)** uses a neural network to approximate Q-values instead of a Q-table.
- Traditional Q-learning uses a table, which is not scalable for large state spaces.
- DQN can handle high-dimensional inputs like images (Atari frames).

2. Challenges in Training DQN & Solutions:

Challenge	Solution
Unstable learning	Use target network
Correlated data	Use experience replay
Overestimation	Use Double DQN
Slow convergence	Tune hyperparameters
Large state space	Use CNNs

Experiment No. 9
To implement policy gradient methods, such as REINFORCE or Proximal Policy Optimization (PPO), to train an agent in a continuous control environment.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: To implement policy gradient methods, such as REINFORCE and Proximal Policy Optimization (PPO), to train an agent in a continuous control environment and analyze their performance in learning optimal policies.

Objective:

- To understand the concept of **policy-based reinforcement learning methods**.
- To implement the **REINFORCE algorithm** for direct policy optimization.
- To implement **Proximal Policy Optimization (PPO)** for stable and efficient learning.
- To train an agent in a **continuous action space environment**.

Theory:

1. Policy-Based Reinforcement Learning

Unlike value-based methods, policy-based methods directly learn a **policy function**:

$$\pi(a \mid s; \theta)$$

Where:

- $\pi(a \mid s; \theta)$: Probability of taking action $a$in state s
- θ: Parameters of the policy

Goal: Maximize expected cumulative reward:

$$J(\theta) = \mathbb{E}[R]$$

2. Policy Gradient Theorem

The policy is updated using gradient ascent:

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \log \pi(a \mid s; \theta) \cdot R]$$

This means:

- Increase probability of actions that yield higher rewards
- Decrease probability of poor actions

3. REINFORCE Algorithm

Concept

REINFORCE is a **Monte Carlo policy gradient method**:

- Uses complete episodes to update policy
- No value function required

Steps

1. Generate an episode using current policy
2. Compute total return R
3. Update policy parameters using gradient

Advantages

- Simple to implement
- Works well for small problems

Limitations

- High variance in updates
- Slow convergence
- Unstable learning

4. Proximal Policy Optimization (PPO)

Concept

PPO improves stability by limiting large policy updates.

Key Idea

- Restrict how much the policy can change in one update
- Uses a **clipped objective function**

$$L^{CLIP}(\theta) = \mathbb{E}[\min(r(\theta)A, \text{clip}(r(\theta), 1-\epsilon, 1+\epsilon)A)]$$

Where:

- $r(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)}$
- A: Advantage function
- ϵ: Clipping parameter

Implementation:

```
import gym

import numpy as np

import tensorflow as tf

from tensorflow.keras import layers


env = gym.make("CartPole-v1")
```

```
state_size = env.observation_space.shape[0]

action_size = env.action_space.n

gamma = 0.99

learning_rate = 0.001



# Policy Network

model = tf.keras.Sequential([

    layers.Dense(24, activation='relu', input_shape=(state_size,)),

    layers.Dense(24, activation='relu'),

    layers.Dense(action_size, activation='softmax')

])



optimizer = tf.keras.optimizers.Adam(learning_rate)



# Training

episodes = 200



for ep in range(episodes):

    state = env.reset()[0]

    states, actions, rewards = [], [], []



    done = False
```

```
    while not done:

        state_input = np.reshape(state, [1, state_size])

        probs = model(state_input).numpy()[0]

        action = np.random.choice(action_size, p=probs)



        next_state, reward, done, _, _ = env.step(action)



        states.append(state)

        actions.append(action)

        rewards.append(reward)



        state = next_state



    # Compute returns

    returns = []

    G = 0

    for r in reversed(rewards):

        G = r + gamma * G

        returns.insert(0, G)



    # Normalize returns

    returns = np.array(returns)
```

```
    returns = (returns - np.mean(returns)) / (np.std(returns) + 1e-8)


    # Update policy
    with tf.GradientTape() as tape:
        loss = 0
        for i in range(len(states)):
            state_input = np.reshape(states[i], [1, state_size])
            probs = model(state_input)
            action_prob = probs[0, actions[i]]
            loss += -tf.math.log(action_prob) * returns[i]


    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))


    print(f"Episode {ep+1}, Total Reward: {sum(rewards)}")



import gym
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers
```

```
env = gym.make("CartPole-v1")


state_size = env.observation_space.shape[0]

action_size = env.action_space.n


gamma = 0.99

epsilon = 0.2

learning_rate = 0.001


# Actor-Critic Model

state_input = layers.Input(shape=(state_size,))

dense = layers.Dense(64, activation='relu')(state_input)

dense = layers.Dense(64, activation='relu')(dense)


action_probs = layers.Dense(action_size, activation='softmax')(dense)

state_value = layers.Dense(1)(dense)


model = tf.keras.Model(inputs=state_input, outputs=[action_probs, state_value])

optimizer = tf.keras.optimizers.Adam(learning_rate)


# Training Loop

episodes = 100
```

```
for ep in range(episodes):
    state = env.reset()[0]
    states, actions, rewards = [], [], []

    done = False
    while not done:
        state_input = np.reshape(state, [1, state_size])
        probs, _ = model(state_input)
        probs = probs.numpy()[0]

        action = np.random.choice(action_size, p=probs)

        next_state, reward, done, _, _ = env.step(action)

        states.append(state)
        actions.append(action)
        rewards.append(reward)

        state = next_state

    # Compute returns
```

```
returns = []

G = 0

for r in reversed(rewards):

    G = r + gamma * G

    returns.insert(0, G)


returns = np.array(returns)


# Training

with tf.GradientTape() as tape:

    total_loss = 0


    for i in range(len(states)):

        s = np.reshape(states[i], [1, state_size])

        probs, value = model(s)

        action_prob = probs[0, actions[i]]


        advantage = returns[i] - value[0]


        ratio = action_prob / (action_prob + 1e-10)

        clipped = tf.clip_by_value(ratio, 1-epsilon, 1+epsilon)
```

```
        loss = -tf.minimum(ratio * advantage, clipped * advantage)

        total_loss += loss


    grads = tape.gradient(total_loss, model.trainable_variables)

    optimizer.apply_gradients(zip(grads, model.trainable_variables))


    print(f"PPO Episode {ep+1}, Reward: {sum(rewards)}")
```

Conclusion:

Policy gradient methods directly optimize the policy and are suitable for continuous action spaces.

- **REINFORCE** is simple but suffers from high variance and instability.
- **PPO** improves performance by stabilizing updates using clipping.

Thus, PPO is widely used in modern reinforcement learning due to its efficiency and reliability.

Experiment No. 10
Case Study on Applying reinforcement learning techniques to solve a real- world problem, such as training a self-driving car to navigate a simulated road environment.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim:

To study and analyze the application of reinforcement learning techniques in solving a real-world problem such as autonomous driving in a simulated environment.

Objective:

- **To understand how reinforcement learning is applied in real-world systems**
- **To analyze the working of RL in autonomous vehicles**
- **To explore algorithms used in self-driving cars**
- **To evaluate challenges and performance**

Theory:

1. Reinforcement Learning in Real-World Problems

Reinforcement Learning (RL) is widely used in real-world applications where an agent learns optimal behavior through interaction with an environment using rewards and penalties.

---

2. Problem Statement: Self-Driving Car Navigation

The goal is to train an autonomous vehicle to:

- **Navigate roads**
- **Avoid obstacles**
- **Follow traffic rules**
- **Reach destination safely**

---

3. MDP Formulation for Self-Driving Car

Component	Description
States (S)	Position, speed, surroundings, sensor data
Actions (A)	Accelerate, brake, turn left/right
Reward (R)	+ve for safe driving, -ve for collision
Policy (π)	Driving strategy
Environment	Simulated road (e.g., CARLA, OpenAI Gym)

---

4. RL Algorithms Used

- **Deep Q-Network (DQN) → Discrete actions**
- **Policy Gradient (PPO) → Continuous control**
- **Actor-Critic Methods → Better stability**

---

5. Working of RL in Self-Driving Cars

1. **Car observes environment via sensors (camera, LiDAR)**
2. **State is fed into RL model**
3. **Model predicts best action**
4. **Action is executed (steer, accelerate, brake)**
5. **Reward is given based on performance**
6. **Model updates policy to improve driving**

Implementation (Conceptual Code Example):

```
import gym

import numpy as np



# Simulated environment (simplified)

env = gym.make("CarRacing-v2")



episodes = 10



for ep in range(episodes):

    state = env.reset()[0]

    total_reward = 0



    done = False

    while not done:

        # Random policy (for demo)

        action = env.action_space.sample()



        next_state, reward, done, _, _ = env.step(action)

        total_reward += reward



        state = next_state
```

```
print(f"Episode {ep+1}, Score: {total_reward}")
```

Real-World Example:

Autonomous Driving Systems (e.g., Tesla-like systems)

- **Use deep neural networks + reinforcement learning**
- **Trained on massive driving datasets**
- **Perform tasks like:**
  - **Lane detection**
  - **Object avoidance**
  - **Adaptive cruise control**

Outcome:

- **RL enables vehicles to learn driving behavior without explicit programming**
- **Improves safety and automation**
- **Adapts to dynamic environments**

Challenges:

- **Safety and reliability**
- **High computational cost**
- **Need for large training data**
- **Real-world uncertainty**

- **Ethical concerns (decision-making in accidents)**

Advantages:

- **Learns from experience**
- **Adapts to new environments**
- **Reduces human effort**
- **Handles complex decision-making**

Conclusion:

Reinforcement learning plays a crucial role in autonomous driving by enabling intelligent decision-making through interaction with the environment. While it offers significant advantages in automation and adaptability, challenges like safety, computational complexity, and real-world uncertainty must be addressed for large-scale deployment.